

Source Code Hotspots: A Diagnostic Method for Quality Issues

Saleha Muzammil
University of Virginia
Charlottesville, Virginia
USA
saleha@email.virginia.edu

Mughees Ur Rehman
Virginia Tech
Blacksburg, Virginia, USA
mughees@vt.edu

Zoe Kotti
Athens University of
Economics and Business
Athens, Greece
zoekotti@aueb.gr

Diomidis Spinellis
AUEB & TU Delft
Athens, Greece
dds@aueb.gr

Abstract

Software source code often harbours “hotspots”—small portions of the code that change far more often than the rest of the project and thus concentrate maintenance activity. We mine the complete version histories of 91 evolving, actively developed GitHub repositories and identify 15 recurring line-level hotspot patterns that explain why these hotspots emerge. The three most prevalent patterns are *Pinned Version Bump* (26%), revealing brittle release practices; *Long Line Change* (17%), signalling deficient layout; and *Formatting Ping-Pong* (9%), indicating missing or inconsistent style automation. Surprisingly, automated accounts generate 74% of all hotspot edits, suggesting that bot activity is a dominant—but largely avoidable—source of noise in change histories. By mapping each pattern to concrete refactoring guidelines and continuous integration checks, our taxonomy equips practitioners with actionable steps to curb hotspots and systematically improve software quality in terms of configurability, stability, and changeability.

ACM Reference Format:

Saleha Muzammil, Mughees Ur Rehman, Zoe Kotti, and Diomidis Spinellis. 2026. Source Code Hotspots: A Diagnostic Method for Quality Issues. In *23rd International Conference on Mining Software Repositories (MSR '26)*, April 13–14, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3793302.3793334>

1 Introduction

A key advantage of software over hardware is that it is considerably more malleable. Yet software changes come at a cost: they require effort to develop, test, and deploy them, and they may introduce faults or lead to failures. Consequently, it is profitable to improve our understanding of why software is changing and our management of software changes.

Inspired by an electrical engineering diagnostic method where infrared thermography is employed to assess equipment reliability [33] and identify defects [28], we propose and study source code hotspots as a way to identify software engineering defects. We term frequent localized changes to specific software source code lines as *source code hotspots* (hotspots from now on).

We posit that when a hotspot appears, it often signals a deeper pathology in the codebase. The root cause can be a weak software architecture, a flawed design, inadequate tooling, or a deficient software development process. Frequently, the underlying issue fits an anti-pattern [7], such as Continuous Obsolescence [7, p. 85] or

Stovepipe Enterprise [7, p. 147]. For example, a hard-coded currency exchange rate will require frequent updates to its value. A better design would have that value stored in a database or configuration file or retrieve it through a web service.

From a theoretical perspective, our research focuses on software quality, particularly on maintainability. Maintainability is a critical attribute encompassing, among others, three interrelated dimensions: *configurability*, *stability*, and *changeability* [67]. *Configurability* denotes the ease with which behaviour can be adjusted via settings or external files, reducing the need for source edits. Lack of configurability forces code modifications for every slight adjustment, inflating code churn. *Stability* measures a system’s resilience to unintended impacts when changes occur elsewhere; unstable systems incur cascading edits across modules. Finally, *changeability* captures how simply modifications (e.g., feature additions or bug fixes) can be implemented; sub par changeability leads to repetitive corrections of the same lines. Software code that changes often may lack in changeability, miss configurability options, or exhibit instability when modified. Deficiencies in any dimension can manifest as hotspots.

An additional challenge is that modern software evolution is increasingly shaped by automation. Bots routinely perform dependency updates, metadata regeneration, and formatting changes, inflating code churn and potentially obscuring developer-driven maintenance effort. In our context, bot activity refers to commits authored by automated accounts recorded in Git history. Without distinguishing such automated changes from human-authored edits, hotspot analyses risk conflating mechanical noise with meaningful software evolution, making it difficult to identify which patterns reflect genuine maintainability issues versus routine mechanical updates.

Unlike previous research that primarily focused on detecting anti-patterns arising from design principle violations at coarse granularity (e.g., files or commits) [13, 50, 60], our work pioneers the identification of *line-level* code churn. We developed and employed an algorithm and a corresponding tool that perform fine-grained analysis of revision histories to track the evolution of individual lines over time. This line-level granularity isolates the exact loci of repeated change, enables actionable diagnosis of distinct churn sources (e.g., configuration constants versus formatting noise), and allows attribution of hotspot edits to human developers versus automated accounts based on committer identity. By zooming in on these frequently altered lines, we can not only relate them to design or process anti-patterns but also pinpoint , avoidable changes. Thus, our identification of hotspots offers a concrete strategy for improving software quality. To achieve these goals, we have formulated the following research questions.



This work is licensed under a Creative Commons Attribution 4.0 International License. *MSR '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2474-9/26/04

<https://doi.org/10.1145/3793302.3793334>

RQ1: Which technical or process factors most often give rise to hotspots, and what concrete practices can curb their recurrence?

RQ2: How do hotspots vary in scope, persistence, and location (e.g., administrative files vs. executable code) across diverse projects?

RQ3: How do bot-authored commits influence hotspots, and what specific anti-patterns are predominantly bot-driven?

This study contributes the following: a novel method for identifying a class of software and process defects based on (line-level) hotspots, a tool for identifying hotspots through the granular analysis and propagation of changes in revision histories; a taxonomy of reasons that give rise to hotspots; metrics regarding the occurrence of hotspots in a stratified sample of open-source software projects; and, an examination of the role bots play in the creation of hotspots.

The remainder of this paper is organized as follows. Section 2 reviews prior work on code churn, change taxonomies, and the role of automation in software evolution. Section 3 describes our methods, including repository sampling, hotspot detection, taxonomy construction, and bot identification. Section 4 presents our hotspot taxonomy, quantitative characteristics of hotspot occurrence, and comparative analysis of bot-generated versus human-authored changes. Section 5 discusses the implications of our findings for software maintainability and development practice. Finally, Sections 6 and 7 outline threats to validity and conclude.

2 Related Work

We organize our review around four key research threads that inform our contributions: (1) the foundational understanding of code churn and source code hotspots in predicting software quality; (2) existing taxonomies of software smells and anti-patterns that motivate our line-level approach; (3) technical advances in mining code churn; and (4) the emerging influence of automated bots on software development practices.

2.1 Code Churn and Fault Prediction

The relationship between code churn and software quality has been extensively studied, establishing churn as a reliable predictor of fault-prone areas. Early work by Graves et al. [26] demonstrated that files with high modification frequencies correlate strongly with defect density, a finding corroborated by Nagappan and Ball [54] and Kim et al. [36] across multiple industrial projects. Hassan [27] advanced this line of research by introducing temporal complexity metrics, showing that the *pattern* of changes matters as much as the volume. Giger et al. [24] analyzed types of code changes, further illustrating that specific churn types can predict future modifications. Beyond defect-focused analyses, the shape of project activity matters: the distribution of commit frequency in open source projects is highly skewed, with a small fraction of contributors and periods accounting for a disproportionate share of commits [39]. This heavy-tailed activity further motivates our investigation of line-level hotspots. Empirical studies have characterized *what* developers change and *why*. For example, work on change-reason analysis uses historical repositories to infer the intent behind modifications [51]. Other studies propose and analyze

taxonomies of change types and patterns [18, 31, 32, 71], including mining unknown change patterns from code histories [55].

However, these studies predominantly are defined at file or module granularity, leaving a gap in understanding the specific *lines* that drive hotspot behavior. Our work addresses this limitation by providing the first systematic analysis of line-level code churn patterns across a stratified sample of repositories and the idea of using hotspots as a diagnostic method for software quality issues.

2.2 Software Smells, Anti-Patterns, and Change Taxonomies

Code smells [20] and design anti-patterns [7, 38, 73] signal maintainability debt. At code level, Fowler’s seminal catalogue (*God Class*, *Long Method*, *Feature Envy*) formalised “smells” as maintainability warning signs [20]. This catalogue has since expanded to encompass architectural anti-patterns [7, 38, 73], with Mo et al. [49] identifying structural problems like *Unstable Interface* and *Modularity Violation*. In this area, Garcia et al. [23] contributed systematic detection approaches for architectural smells, Sas et al. [63] identified co-changes as a way to detect architectural smells, and Mo et al. [48] proposed explicitly the employment of hotspot patterns to identify architecture smells.

Modern software development has introduced new categories of smells. Taibi and Lenarduzzi [70] catalog microservice-specific anti-patterns, Sharma et al. [64] identify configuration smells, showing that configuration smells exhibit higher co-occurrence than implementation smells, while Picha et al. [62], in common with our goals, describe the detection of process anti-patterns from project data.

Parallel to structural smell research, change-focused taxonomies have emerged. Early frameworks by Mens et al. [45] and Buckley et al. [8] established theoretical foundations for classifying software evolution patterns. More recent work has provided empirical taxonomies [31, 42, 61].

Importantly for our work, Nguyen et al. [57] demonstrated that small code changes exhibit high repetitiveness (70–100%), with patterns remaining consistent both within and across projects. This finding directly motivates our line-level analysis, as repetitive change patterns suggest systematic maintainability issues that can be addressed through targeted interventions. Most existing smell and change taxonomies operate at coarse granularity (e.g., classes, files, components, or services) [8, 31, 42, 45, 61]. While useful for describing broad maintenance trends, such abstractions can conflate heterogeneous edits within the same artifact, limiting actionability. In contrast, our contribution provides a line-level taxonomy, which distinguishes pathological churn from semantically meaningful edits and thereby enables direct CI enforcement and the proposal of concrete refactoring guidelines.

2.3 Techniques and Tools for Detecting Code Churn

Textual difference algorithms have traditionally been employed to capture software changes at various granularities [47, 53]. Modern approaches [16, 17] offer significantly improved accuracy and granularity, facilitating the identification of specific line-level changes and thus enabling hotspot detection [3].

Tools built on these methods like ChangeDistiller [19], utilize tree-differencing algorithms to extract source code changes efficiently. Similarly, PYCT [43] extends change detection capabilities to Python codebases, reducing manual overhead in classification tasks.

While various tools exist for architectural and design anti-pattern detection (e.g., SonarQube, Designite [65], DV8 [9]), there remains a notable absence of tools explicitly designed for identifying line-level code churn patterns (hotspots) from revision histories. Addressing this gap is central to our research.

2.4 Automation and Bots

Large-scale profiling of GitHub activity shows substantial heterogeneity in who contributes, what changes are made, and when/where activity concentrates across ecosystems, with an increasing involvement of bots [75]. Building on this macro view, we focus on the automation slice of that activity, examining how bot-generated changes interact with churn and hotspots. Automation now generates a non-trivial share of source code commits. Bots automate a range of tasks, often leading to superficial or repetitive changes that inflate churn metrics and potentially obscure substantial development efforts [15].

Wessel et al. [74] explored code-review bots, demonstrating that while bots streamline workflows by merging pull requests more rapidly, they simultaneously reduce developer engagement and collaborative interactions, potentially hindering meaningful software evolution. Similarly, Dey et al. [11] found that bot-generated minor changes frequently overwhelm change histories, masking important developer-driven modifications and inflating overall churn.

Furthermore, Kinsman et al. [37] investigated GitHub Actions, identifying an increase in pull request rejections and a reduction in merged commits post-adoption, further illustrating bots' complex influence on software development practices. Golzadeh et al. [25] developed methods to systematically identify bots through behavioral analysis, reinforcing the importance of distinguishing automated activities from human-driven contributions.

In summary, prior work has (i) measured churn and change patterns largely at file, module, or commit granularity [24, 26, 27], (ii) catalogued smells and anti-patterns and, in some cases, connected them to evolution signals using change-history information [58, 59], and (iii) analyzed bot activity and its effects on development traces [11, 15, 74]. Our study complements this literature by systematically identifying recurring *line-level* hotspot patterns, mapping them to a taxonomy with actionable mitigations, and quantifying the automation share within each pattern.

3 Methodology

Our study aims to identify in open-source software repositories line-level code churn (source code hotspots): files and corresponding lines that change much more frequently than others. To optimize the identification of hotspots we track modifications first at the file level and then at the line level.

3.1 Repository Selection and Mining

Given the more than 420 million repositories hosted on GitHub as of January 2023 [14], analyzing them all was clearly infeasible. We therefore selected projects using stratified sampling [56] based on GitHub stars as a proxy for community interest [5] and GitHub forks as a proxy for direct developer involvement to strike a balance between breadth and tractability. Without stratification, a purely random draw would be dominated by dormant or toy projects [34], whereas focusing only on the most-starred repositories would conflate hotspot phenomena with the engineering cultures of a small elite. Guided by best-practice recommendations for repository curation and mining [10, 34, 52, 72] and building on a specific method employed by Spinellis et al. [69], we divided the population into five strata (11–100, 101–1000, 1001–10 000, 10 001–100 000, and 100 001–1 000 000 stars/forks) and then applied the following inclusion criteria for selecting the repositories.

- More than ten GitHub stars or forks, to select projects relevant to the software engineering community, and avoid personal projects and student exercises. These two metrics are considered to be the most useful GitHub project popularity metrics [5].
- At least one commit on every half-year interval, in order to examine continuously evolving software, with changes hotspots potentially surfacing over longer periods.
- At least 10 000 commits to provide a reasonable opportunity for hotspots to occur.

The metadata of the repositories was queried through the GitHub API on October 20, 2024. To validate our inclusion and exclusion criteria, we manually inspected a random sample of ten repositories from our initial selection. This inspection confirmed that all sampled repositories matched our intended criteria regarding commit counts, creation dates, and fork status.

Our final dataset consisted of 91 repositories, spanning diverse sizes and activity levels; see the rainclouds [1] in Figure 1 depicting metric distributions, inter-quartile ranges, medians, and the actual data points. The long tails in commits, size, and committers are the expected result of the employed stratified sampling. Repository ages ranged from 0.04–16.53 years, with commit counts ranging from 10 114–1 310 015. The number of source code lines ranged from 5 805–38 433 291 lines, while contributor counts ranged from 63–35 543. The repositories represented a wide range of programming languages (see Figure 2), with C++ being the most prevalent (16 repositories), followed by Python (12 repositories) and TypeScript (9 repositories). Java, JavaScript, and C each contributed seven repositories. Other languages including Go, Ruby, specialized languages such as Jinja and PHP, and emerging systems languages like Rust were also represented, ensuring coverage across different programming paradigms and domains. Table 1 summarizes the domain distribution of the analyzed repositories. We assigned each repository to a FreeBSD ports-style category based on its primary purpose (as indicated by the repository description and documentation) and then mapped categories to a small set of descriptive domain tags (e.g., `www` → Web, `sysutils/net` → Systems/DevOps) to improve interpretability.

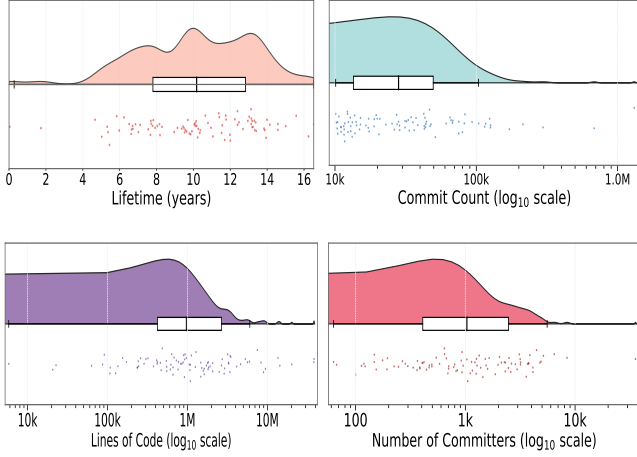


Figure 1: Lifetime (top left), commits (top right), size (bottom left), and contributors (bottom right) of analyzed repositories. We use a $\log_{10}$ scale for commit count, lines of code, and contributor count to visualize heavy-tailed distributions.

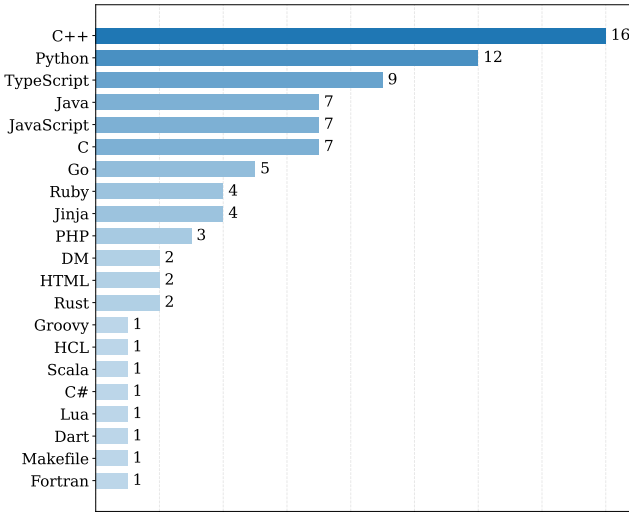


Figure 2: Language Distribution of Projects

3.2 File-Level Analysis

An automated script processed each repository to identify files with high modification frequencies. First, files were categorized into programming files (source code) and administrative files or operational data (e.g., documentation, configuration) [29, p. 241] based on their extensions and filenames. Then, for each file, the script counted the number of commits in which the file was modified. Finally, statistical measures such as the mean and standard deviation of modification counts were calculated.

We identified file-level hotspots through a dual-filter approach designed to minimize false positives while capturing genuine churn centers. Our method employed two complementary outlier detection criteria applied in conjunction. First, we flagged files whose

modification counts exceeded three standard deviations above the mean ($\mu + 3\sigma$). Second, following the approach suggested by Mu-naiah et al. [52], we required each file’s total changes to exceed one change per month of project lifetime ($\text{lifetime_months} \times 1$) to eliminate low-activity files from consideration. Only files satisfying both conditions were classified as potential hotspot hosts, as preliminary analysis revealed that either criterion applied independently produced excessive false positives. While tightening these thresholds—either by increasing the standard deviation cutoff or raising the monthly change rate—enhanced precision, it risked excluding legitimate centers of code churn.

3.3 Line-Level Analysis

To identify hotspots, we performed fine-grained line-level analysis on hotspot files. The task has two parts: (i) counting how often a concrete line changes and (ii) following that same line forward in time even as its position gets shifted by other changes/commits in the file.

We relied on Git to emit consecutive, patch-ordered diffs:

```
$ git log -M -C --pretty=format:'commit %H %ct' \
--reverse -p [file_path]
```

The `-M` flag enables move detection, allowing Git to track when code blocks are relocated. The `-C` flag enables copy detection, identifying when code is duplicated across files. The `--reverse` flag ensures commits are processed in chronological order from oldest to newest. The `-p` flag ensures that the diff is printed in patch format, which is useful for our line-level analysis.

3.3.1 Line-Modification Counting Method. Based on an existing peer-reviewed tool that determines the lifetime of fine-grained elements by tracking specific lines across modifications [68], we developed a line change tracking algorithm (Algorithm 1) that maintains, for every file, `file_lines`, a vector of canonical line objects (including content, commit id, modification count, birth timestamp, etc.), and `line_offsets`, a cumulative map from original to current indices. When a modification appears in a line difference hunk reported by Git, the algorithm:

- (1) adjusts the raw hunk coordinates by the running offset,
- (2) pairs deletions with additions when their positions align, copying the *birth* timestamp and incrementing *mod_cnt*, and
- (3) records any unmatched deletion (resp. addition) as a *death* (resp. new *birth*).

If a line is unchanged in successive commits, its lifespan continues uninterrupted and its identifier (the slot in `file_lines`) remains stable.

Line-identity model and offset adjustment. The `line_offsets` map is what lets us “follow” the same logical line when earlier edits push it up or down. After every processed hunk we update the offset for all subsequent line indices.

Handling moved lines. We deliberately do not attempt to recognise intra-file moves. Once a block disappears from its original coordinates, the tracker marks those lines as dead; re-appearing identical text elsewhere is logged as newly added lines. This positional-identity assumption keeps the algorithm lightweight and deterministic, at the cost of missing some potential hotspots.

Algorithm 1 Git Diff Line Modification Tracker

```

1: Initialize data structures: file_lines, line_offsets, max_processed_index
2: Set state ← “commit”
3: while not end-of-input do
4:   if state = “commit” then
5:     if current_commit exists then
6:       Finalize previous commit and reset tracking data
7:     if line matches “commit <hash> <timestamp>” then
8:       Extract commit_hash and timestamp
9:       Read next lines until encountering “diff”, next “commit”, or EOF
10:      Update state accordingly
11:   else if state = “diff” then
12:     if line matches “diff -git a/<old> b/<new>” then
13:       Extract old_file and new_file paths
14:       if files are in target list then
15:         Initialize tracking for these files if needed
16:         Skip header lines until reaching “@@” line
17:         state ← “range”
18:       else
19:         Skip to next diff or commit
20:   else if state = “range” then
21:     Parse hunk header “@@ -X,Y +A,B @@”
22:     Calculate adjusted file positions using line_offsets
23:     Check for overlap with previously processed hunks
24:     Collect all lines in current hunk
25:     Process hunk:
26:     old_idx ← old_start
27:     updated_lines ← []
28:     for each sequence of lines in hunk do
29:       if context line (starts with “”) then
30:         if overlap > 0 then
31:           Skip and decrement overlap
32:         else
33:           Copy existing line to updated_lines
34:       else if modification (sequence of “-” lines followed by “+” lines) then
35:         Group consecutive deletions and additions
36:         for each pair or unpaired line do
37:           if paired (modification) then
38:             Create new line with updated content
39:             Copy birth timestamp from old line
40:             Increment mod_count
41:             Append to content, commit, and timestamp histories
42:           else if deletion only then
43:             Mark old line with death_ts = current timestamp
44:           else if addition only then
45:             Create new line with birth_ts = current timestamp
46:       Replace affected lines in file_lines with updated_lines
47:       Update max_processed_index to track processed areas
48:       Determine next state based on next line
49:   else if state = “EOF” then
50:     Break main loop
51:   Read next line if needed
52: Output results:
53: for each line info in file do
54:   if line has no death timestamp (still alive) then
55:     Write to CSV: line number, content, mod_count, and history details

```

3.4 Taxonomy Development

To develop a taxonomy of hotspots, we employed a systematic manual labeling process complemented by similarity detection techniques. This approach ensured that the taxonomy accurately reflects the diverse types of line modifications observed in the data.

3.4.1 Manual Labeling Process. Two of this paper’s authors independently reviewed the files generated from line change data. For each hotspot, both researchers first documented their individual interpretations of what was causing the changes, including their reasoning and observations. After this independent documentation phase, the researchers convened to compare their findings, discuss their reasoning, and assign an appropriate tag that best described the nature of the modification. This two-step process—individual documentation followed by collaborative convergence—ensured both independent analysis and consistency in the final labeling, following established systematic review practices where reviewers work independently before resolving disagreements through discussion [6].

To determine labeling saturation, we applied the Chao1 estimator [66], which has been adopted in several prior software engineering studies [4, 44, 46]. It is a non-parametric lower-bound estimator of species (here hotspot-types) richness and is defined as:

$$S_{\text{est}} = S_{\text{obs}} + \frac{f_1^2}{2f_2} \quad (1)$$

In this context, S_{obs} is the number of distinct hotspot types actually observed in our sample, f_1 is the number of hotspot types seen exactly once (singletons), and f_2 is the number of hotspot types seen exactly twice (doubletons).

In Figure 3, the Chao1 estimate rises quickly as new hotspot types appear, then levels off, showing diminishing returns from further labeling. After about 100 assignments the observed count stabilizes at 15 unique hotspot types and stays flat through 160. The Chao1 lower bound remains slightly above this and ends at 15.67 at 160 assignments, a difference of about 0.67 (approximately 4.4 percent). This plateau indicates that the sample is saturated, so additional labeling would likely uncover at most one more label at disproportionate cost. This statistical approach provided a systematic stopping criterion for our manual labeling process, allowing us to make a reasoned judgment about when we had identified a representative sample of the hotspot types present in our dataset.

We manually labeled and investigated the mined hotspots, allowing us to label only a fraction of our mined ones until the number of defined labels reached the goal established by the Chao1 estimator. Our experiment did not aim to label all hotspots; instead, our goal was to describe all major hotspot types.

After labeling, the results were consolidated. If both researchers assigned the same label, that label was considered final. If the labels differed, the researchers discussed their reasoning and decided on a label accordingly. If they were unable to reach a consensus, the paper’s last author — a leading (R4) researcher [21] with considerable software engineering expertise — was consulted, and a tag was mutually decided upon. The inter-rater agreement of the two initial software engineering researchers using Cohen’s Kappa was found to be 0.61, which denotes a substantial level of agreement between the two raters [41].

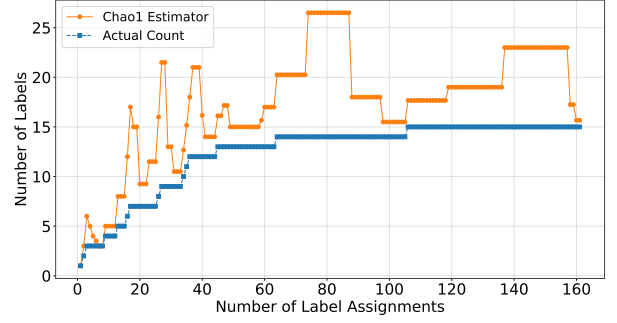


Figure 3: Chao1 Estimator vs. Number of Label Assignments

Table 1: Repository Domain Distribution

Domain tag	FreeBSD category	Count
Web	www	19
Systems/DevOps	sysutils	14
Dev tools & SDKs	devel	10
Miscellaneous	misc	7
Games	games	6
Science/Engineering	science	5
Python Packages	python	4
Languages	lang	4
Mathematics	math	3
Systems/DevOps	net	3
Docs/Content	textproc	3
Dev tools	editors	3
Java Packages	java	2
Graphics	graphics	2
Databases	databases	2
Desktop/UI	x11	2
Finance/Crypto	finance	1
Security	security	1

3.5 Bot Detection

To identify bots that contributed to hotspots, we analyzed commit data. For each hotspot line, we aggregated the commit count made by each contributor. This was achieved by grouping commits based on the committer’s e-mail and name, which we extracted using the `git show` command on the relevant commit IDs.

We consolidated all committer information into a single file. To do this, we grouped contributors with the same name and e-mail, then applied case-insensitive regular expressions to flag usernames or display names containing keywords like “bot” or “auto.” To ensure high accuracy, we manually reviewed every flagged entry and identified just one false positive—“Drobotov,” whose name inadvertently matched our filter. This process allowed us to pinpoint genuine bots and assess their significant impact on code-churn.

By aggregating contributor names or e-mails associated with hotspots across all projects and analyzing the most frequent ones, we could effectively identify and isolate bot activity.

4 Results

This section presents our identified hotspot types, their characteristics, and the role of automated contributions in generating these patterns.

4.1 Identified Hotspot Types (RQ1)

Through our analysis, we discovered and characterized a total of 15 unique hotspot types, which have been divided into four umbrella categories. Each hotspot type is described below, along with a representative example demonstrating the changes' nature. All colored code lines are hyperlinked to permanent Software Heritage archive [12] locations using the ISO/IEC 18670:2025 standard for SoftWare Hash IDentifiers (SWHIDs) [30]. We also identified and classified instances of normal *software evolution* which are legitimate and can appear as frequent edits but do not represent anti-patterns.

4.1.1 Configuration Management Hotspots. These include changes to version specifications, dependencies, and resource configurations that govern software component interactions and deployment.

Pinned Version Bump involves incrementing a fixed (pinned) version in the absence of flexible or semantic versioning strategies.

```
- release = "3.10.181"
+ release = "3.10.182"
```

Frequent changes in pinned versions can be avoided by using version ranges instead of fixed version pins. We also suggest avoiding committing files which contain locked versions (e.g., `pipfile.lock`, `package-lock.json`). Semantic versioning can also help us here by updating versions selectively when there's a meaningful change (bug fix, feature, or breaking change). This contrasts with arbitrary version bumping that might happen on every release cycle regardless of significance.

Conditional Version Bump involves adjusting conditions related to version checks.

```
- numpy>=1.15,<1.19.0
+ numpy>=1.16.5,<1.19.0
```

The same mitigation strategies as for Pinned Version Bump can be followed to avoid frequent conditional version bumps.

Resource ID Modification involves changes to resource identifiers such as Docker images or Amazon Machine Image (AMI) IDs. Small oversights in these references can lead to deploying incorrect or deprecated resources.

```
- IMAGE=container-vm-v20141208
+ IMAGE=container-vm-v20150112
```

To minimize these changes, provide the resource IDs as command-line parameters, or retrieve them through APIs using stable tags, or store them in environment variables.

Service Configuration involves modifications to service configuration, including SSH tokens, IP addresses, and port numbers.

```
- sunbird_user_service_api_base_url:
  "http://{{sunbird_swarm_manager_lb_ip}}:9000"
+ sunbird_user_service_api_base_url:
  "http://{{private_ingressgateway_ip}}/learner"
```

To avoid these changes, move external service configurations to environment variables or command-line arguments. This also prevents sensitive information from appearing in hotspots. It is also suggested to use tools like Terraform, Ansible, or Kubernetes with secret management solutions (HashiCorp Vault, AWS Secrets Manager, etc.) and define configurations declaratively.

Dependency Specification involves adjustments in dependency declarations, such as adding, removing, or modifying modules in import statements or dependency arrays, or updating entries in a package.json, pom.xml, or similar file.

```
- obj-$(CONFIG_VIDEO_DEV) += videodev.o compat_ioctl32.o
  v4l2-int-device.o
+ obj-$(CONFIG_VIDEO_DEV) += videodev.o v4l2-compat_ioctl32.o
  v4l2-int-device.o
```

To avoid frequent dependency specification changes, implement a robust dependency management strategy. Automated dependency update tools (e.g., Dependabot, Renovate) can reduce manual version edits by systematically batching and standardizing dependency updates. However, without appropriate configuration, such tools may generate excessive update churn through overly frequent or fragmented updates. Controlled update policies (e.g., scheduled batched updates, grouped dependency bumps) can mitigate this risk while maintaining security currency.

External Data Fluctuations are driven by external data sources, such as including updated currency exchange rates in a database or regenerating code from an external API specification.

```
- tier: "NU",
+ tier: "PU",
```

To remove external data fluctuations, obtain the corresponding data automatically from stable interfaces for external data sources. The performance impact can be minimized by employing caching mechanisms that reduce the update frequency and impact.

4.1.2 Development Environment Hotspots. These include modifications to development infrastructure including file paths, base distributions, and debugging settings that affect the build environment.

Path Update occurs when file paths or imports are altered, often due to project restructuring or changes in directory organization. This also includes modifications made to URL links.

```
- import { IProjectStats } from 'lib/services/project-service';
+ import { IProjectStats } from '../services/project-service';
```

To avoid frequent path updates, it is suggested to structure the project according to established guidelines such as those by Maven. In addition, projects should employ *stable import aliases* that decouple logical module names from their physical directory layout. Mechanisms such as TypeScript path mappings, bundler aliases (e.g., Webpack or Vite), or equivalent language-specific solutions allow directory reorganization without requiring widespread import rewrites. Abstracting imported resources and URL specifications into configuration files further minimizes churn and reduces the risk of broken imports during structural changes.

Distro Bump refers to upgrading or changing the base distribution (e.g., Linux distribution, container base image) to a newer release.

```
- name: kolla-ansible-centos8s-source-kvm
+ name: kolla-ansible-rocky9-source-kvm
```

To mitigate the need for frequent distro bumps, choose a stable and long-term supported base distribution.

Debug Configuration refers to changes in debugging or logging settings (e.g., adding `-v` or `-r` flags to commands). While often required for troubleshooting, leaving excessive debugging configurations in production can introduce overhead or leak sensitive data.

```
- extra_var_arg+= ' -e instance_userdata="" -e launch_wait_time=0'
+ extra_var_arg+= ' -e instance_userdata="" -e launch_wait_time=0 -e
  elb_pre_post=false'
```

To avoid frequent debug configuration changes, debugging and logging parameters should be externalized into environment variables or feature-flag frameworks rather than committed directly to source files. Tools such as environment-variable loaders can support this separation, while continuous-integration checks can detect and flag accidental commits of debug-specific settings, preventing unnecessary churn.

4.1.3 Code Quality and Style Hotspots. These include formatting adjustments, function signature changes, and structural improvements that enhance code readability without altering functionality.

Function Call Change encompasses alterations in function signatures (parameters, return types) or renaming of method calls.

```
- printf("%s", find_unique_abbrev(get_object_hash(parent->object),
  abbrev));
+ printf("%s", find_unique_abbrev(parent->object.oid.hash, abbrev));
```

To minimize the need for frequent function call changes, define stable interfaces for core functions and create stable wrapper functions or adapters. In languages that support abstract classes and interfaces, (such as Java, C#, TypeScript, and Kotlin), follow the robustness principle [2] by passing arguments as abstract class types and returning values as interface types. In languages that support keyword argument passing, (such as Python, Ruby, Kotlin, Swift, Scala, Julia, and Elixir), employ named rather than positional arguments. In most object-oriented languages pass arguments as object, use the builder pattern [22, p. 97], or employ polymorphism.

Formatting Ping-Pong represents purely stylistic or formatting-related changes, such as adjusting whitespace, using tabs, or toggling between uppercase and lowercase. These changes can clutter commit histories and obscure more critical modifications.

```
- #include <assert.h>
+ #include <assert.h>
```

To minimize Formatting Ping-Pong, automated linters and code formatters (e.g., Prettier, Black, gofmt, clang-format, etc.) should be used. CI/CD pipelines can also be configured to validate formatting compliance with an agreed upon standard.

Long Line Change captures edits to long lines of text or code, such as extensive comments, documentation paragraphs, long import statements, or expressions. The length of modified lines often

obscures the actual changes in the output of common line-oriented diff algorithms.

```
- On the right side of the canvas is Search, and the Global Menu.
  You can use Search to easily find components on the
```

```
+ On the right side of the canvas is Search, and the Global Menu.
  For more information on search refer to «search». The Global Menu
```

To minimize long line changes, break down long lines into smaller ones. This can include refactoring long lines of code into smaller functions or methods. In the documentation, employ *semantic line breaks* [35, p. 11], starting each phrase in a new line and breaking long lines near 60–70 columns. Linting rules should be implemented that flag when strings exceed a certain length. We also suggest creating pre-commit hooks to enforce these practices.

4.1.4 Administrative Hotspots. These include updates to non-functional metadata and documentation for legal compliance and project management.

License Modification refers to updates in copyright headers, entity names, years, or licensing text. While these changes are often boilerplate, they need to be minimized or performed accurately to maintain legal clarity.

```
- # Copyright (C) 2008-2013 TrinityCore <http://www.trinitycore.org/>
+ # Copyright (C) 2008-2014 TrinityCore <http://www.trinitycore.org/>
```

To avoid frequent license modifications, simplify license maintenance processes, e.g., by not updating the copyright year. Rather than having the whole license in each file, only provide a reference to a single shared license file.

Metadata Change involves updating automatically generated information such as timestamps, signatures, checksums, or version identifiers. These changes can be misleading if included in commits without actual content changes, as they inflate the perceived scale of a revision.

```
- "timestamp": "2020-10-03T12:37:57.000+00:00",
+ "timestamp": "2021-01-25T14:24:58.697Z",
```

To avoid frequent metadata changes, reduce the amount of metadata stored under revision control by automating their generation during the build process. Use build tools like Maven, Gradle, or `setuptools-scm` to derive version information from Git tags rather than hard coding metadata in source code files. Adopting the goal of reproducible software builds [40], will reduce needless metadata and thus corresponding changes, while also increasing the software’s trustworthiness.

Stepwise Refactoring reflects code modifications intended to restructure or improve readability without affecting the external behavior. The modifications made in this hotspot type are closer in time. While code refactoring is essential, frequent small-scale changes may clutter the commit history, making it harder to track significant modifications.

```
- menu_opts = model_title.starts_with?("Parent") ? {} : {:menu =>
  chart[:menu], :zoom_url => zoom_url}
+ menu_opts = parent ? {} : {:menu => chart[:menu], :zoom_url =>
  zoom_url}
```


Stepwise refactoring is often necessary to safely address architectural and design problems. Hotspots arise when refactoring proceeds through many small, mechanically repetitive edits to the same lines, inflating churn without providing globally useful intermediate states. To reduce version-history noise while preserving safe refactoring practices, developers can (i) maintain explicit refactoring plans with defined invariants, (ii) use short-lived branches and squash intermediate commits before merging, and (iii) employ automated refactoring tools where available. This mitigation targets version-history noise rather than discouraging incremental refactoring itself.

4.2 Hotspot Characteristics (RQ2)

Our analysis indicates a significant distribution between changes in administrative and programming files across the identified hotspots (see Figure 4). Some repositories only had hotspots in administrative files and not in the source-code files. Therefore, administrative changes account for 53.6% of all the identified hotspots.

Many hotspot types stem from administrative files: *Pinned Version Bump*, *Conditional Version Bump*, *Long Line Change*, and *Metadata Change*. In contrast, *Function Call Change*, *Debug Configuration*, *Resource ID Modification*, and *Path Update* show a stronger programming orientation. This split highlights how understanding the nature of changes can guide different intervention strategies. For example, addressing administrative hotspots may require improved configuration management systems, while programming-dominated hotspots might benefit from design and architectural refactoring (e.g., interface stabilization and modularization) to localize change.

Table 2 shows that hotspot activity exhibits extreme skewness across our dataset. Among 91 repositories, the total file count was 3 277 296, with 28 470 outlier files (0.87% of all files) containing at least one hotspot line. This indicates that hotspots are relatively rare, yet they can have a significant impact on the overall codebase. Among 174 randomly selected hotspot-bearing files, we identified 4 052 outlier lines with a highly skewed distribution: while the median file contains only seven outlier lines, the mean reaches 23.3,

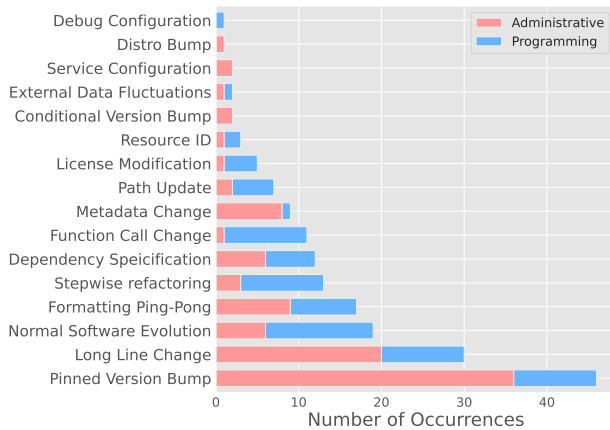


Figure 4: Distribution of Occurrences per Hotspot Type

Table 2: Descriptive statistics for hotspots (4 052 lines drawn from 174 files).

	Min	Median	Mean	Max	IQR
Hotspot lines/file	1.00	7.00	23.29	491.0	21.75
Lifespan (years)	0.00	3.28	4.29	18.13	3.83
Modification Count	3.00	11.0	87.40	1 864	32.0

and one extreme file harbors 491 outliers. Analysis of outlier line lifespans (measured as the temporal difference between first and last modifications) demonstrates remarkable longevity: half persist for ≥ 1 198 days (≈ 3.3 years), the upper quartile survives beyond 2 064 days (5.6 years), and the most persistent hotspot endured 18.1 years. Regarding editing intensity, the median hotspot undergoes eleven modifications, yet exhibits heavy-tailed behavior with 25% of hotspots exceeding 37 edits and the most pathological line experiencing 1 864 changes. These empirical findings substantiate our hypothesis that a small subset of lines concentrates a disproportionate share of long-term maintenance effort, thereby warranting specialized treatment in our analytical framework.

4.3 Bot Activity (RQ3)

Our analysis identified automated bots as significant contributors to hotspots across the studied projects. In total, 23 unique bots were identified (representing 2.8% of the 825 total hotspot committers across all studied projects). Table 3 lists the ten most active bots, with *skia-flutter-autoroll* being the most prolific, responsible for 10 541 commits. Collectively, these bots accounted for 15 283 out of 20 690 total analyzed commits (approximately 73.9%).

Figure 5 depicts a distribution of automation across hotspot categories. Five patterns (*Stepwise Refactoring*, *Service Configuration*, *Resource ID*, *Conditional Version Bump*, and *Distro Bump*) are entirely human-driven (100% human commits). Three more (*Normal Software Evolution*, *Dependency Specification*, and *Long Line Change*) show only small traces bot activity ($< 1\%$). Conversely, bots generate nearly a third of all *Debug Configuration* (29%) and *Path Update* edits (30%). *Formatting Ping-Pong* and *Pinned Version Bump* are almost an even split (46% and 52% bot commits, respectively). The most extreme case is the *Metadata Change*, where bots account for 95% of the edits.

This pattern confirms that currently bot automation concentrates on mechanically repetitive tasks (semantic-version bumps, formatting tweaks, and metadata refreshes), while changes that require domain knowledge (e.g., function-call refactoring or external-resource updates) remain overwhelmingly human. From a maintainability standpoint, these findings suggest that maintenance efforts should focus on (i) suppressing bot-generated noise in code-review workflows and (ii) highlighting human-authored hotspots (which still dominate many error-prone areas) as the primary targets for reviewing and refactoring.

The substantial proportion of bot-driven commits underscores their pivotal role in generating frequent, often repetitive changes. This finding aligns with prior research emphasizing the influence of bots on code churn [11]. For academia, these results highlight the

Table 3: Top Ten Identified Bots

Bot Name	Commit Count
skia-flutter-autoroll	10541
vercel-release-bot	1044
Electron Bot	795
dependabot[bot]	530
jenkins-x-bot	483
GitHub Actions Bot / github-actions[bot]	441
Sudowoodo Release Bot	438
Confluent Jenkins Bot	130
Protobuf Team Bot	128
Netty Project Bot	124

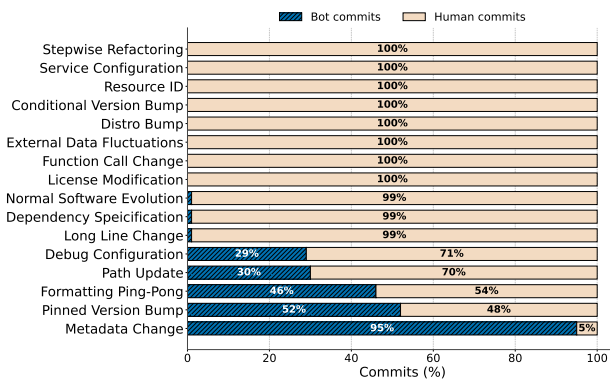
necessity of differentiating automated contributions from human-driven development efforts when conducting repository analyses. In industry, recognizing the impact of bots can guide more effective metric interpretations, resource allocation, and optimization of automated workflows. Open-source software communities benefit by understanding these patterns, enabling maintainers and developers to fine-tune bot behaviors, mitigate unnecessary churn, and focus human attention on meaningful software development activities.

5 Discussion

This section interprets our results through the lens of the three research questions (RQs) and the maintainability dimensions of *configurability*, *stability*, and *changeability*. We also discuss implications for practitioners and future research.

5.1 What factors give rise to hotspots and how can they be curbed? (RQ1)

Our taxonomy uncovered fifteen recurring patterns. Three of them explain almost half of all hotspot edits: *Pinned Version Bump*, *Long Line Change*, and *Formatting Ping-Pong*. All three stem from tooling or process deficiencies rather than inherent domain volatility, confirming that a significant portion of hotspots is preventable. These patterns suggest three key principles for reducing hotspots:

**Figure 5: Bot vs. Human Commit Ratio**

Configurability Pinned or conditional version bumps and resource-ID tweaks surface because values are committed rather than externalised. Moving these to declarative manifests or environment variables would eliminate the need for code edits.

Stability Frequent path rewrites and distro bumps indicate fragile directory structures and infrastructure baselines. Adopting stable import aliases and long-term-support base images would dampen ripple effects.

Changeability Formatting oscillations and long-line rewrites clutter histories and reviews. Enforcing automated formatters in CI avoids human “ping-pong”.

These observations translate directly into actionable guidelines as provided in our paper.

5.2 How do hotspots vary in scope, persistence, and location? (RQ2)

Hotspot activity is *extremely* skewed: a median of seven outlier lines per file contrasts with a mean of 23.3 and a maximum of 491 (Table 2). Lifespans are long (median 3.3 years; max 18.1 years) and editing intensity is heavy-tailed (median 11 edits; max 1 864). These numbers corroborate the Pareto-like intuition that a tiny fraction of lines soaks up a disproportionate maintenance budget.

Location matters. More than half of the hotspots live in administrative files. Patterns such as *Pinned Version Bump* and *Metadata Change* are almost exclusively administrative, whereas *Function Call Change* skews towards executable code. Countermeasures must therefore differ: configuration-management tooling for administrative hotspots versus refactoring and architectural hardening for code hotspots.

5.3 How does bot activity influence hotspots? (RQ3)

Bots account for 73.9% of the 20 690 hotspot-related commits, originating from just 23 distinct automated accounts (Table 3). Their influence is pattern-specific: *Metadata Change* is 95% bot-generated, while five patterns (e.g., *Service Configuration*, *Stepwise Refactoring*) are entirely human. Bots therefore concentrate on mechanical, high-frequency edits (semantic-version bumps, formatting tweaks), yet humans remain dominant in semantically rich or context-dependent changes. Filtering bot commits in dashboards and code-review tools would reduce noise and help teams focus on truly error-prone regions.

5.4 Practice and Research Implications

Practitioners can operationalize our taxonomy by translating recurring hotspot patterns into automated CI checks. A practical workflow is to periodically run our hotspot detector to identify dominant hotspot types, map these types to targeted CI rules, and enforce these rules on incoming pull requests to prevent avoidable churn. For example, if developers observe repeated *Formatting Ping-Pong* or *Dependency Specification* changes, they could decide to refactor or establish more stable standards in the development process. Teams can then track whether hotspot frequency decreases

over time, providing quantitative feedback on mitigation effectiveness.

Future research could focus on empirically quantifying the impact of specific hotspot types on software quality metrics, such as defect rates and maintenance costs. Additionally, integrating machine learning techniques to automatically detect these hotspot types could provide developers with real-time insights, facilitating more informed decision-making in code management. Furthermore, it is important to understand the role of automated bots as hotspot generators. As highlighted by Erlenhov et al. [15], bot-driven changes can sometimes introduce superficial modifications that inflate churn metrics, potentially masking significant development activities.

6 Threats to Validity

Internal Validity. Our line modification counting algorithm can miss or misrepresent some changes. During our manual labeling process we counted such changes—primarily occurring in very large and complex *diffs*. Through this process we established that our algorithm has an accuracy of 90%, with 16 misreports out of 166. Additionally, our tracker defines identity strictly by file path and line index. If developers move a block without editing its text, the original lines are logged as deleted and new ones as created, which shortens apparent lifespans and modification counts. Prior work based on tree-differencing can recover such moves [16], but mainstream *git diff* does not expose that information at line granularity.

Git log limitations may also affect the precision of our data extraction. Our operationalization of *source code hotspots* as lines with modification counts exceeding three standard deviations above the mean, while aligned with established outlier detection approaches, represent a specific definition. Alternative thresholds or time-weighted modification frequencies might identify different sets of hotspots.

External Validity. In our study we mined repositories exclusively from GitHub, relying on GitHub's revision history. This dependency might limit the applicability of our findings to other version controls systems. Despite this limitation, our approach for identifying hotspots can still provide valuable insights about how developers make code changes across different development environments and platforms.

7 Conclusion

In this study, we have identified and categorized source code hotspots (line-level instances of code churn) in open-source software. Through our analysis of 91 diverse GitHub repositories, we discovered 15 distinct hotspot types. Our findings provide two insights that challenge conventional assumptions about code churn. First, more than half of identified hotspots occur in administrative files rather than source code, indicating that configuration management practices significantly contribute to unnecessary revisions. Second, bot activity accounts for 73.9% of all analyzed commits, demonstrating the substantial impact of automated processes on code modifications.

Hotspots may indicate issues associated with software changeability, configurability, and stability—key quality attributes that

directly influence software maintainability. By understanding these patterns, developers can proactively address issues that lead to frequent, avoidable changes in their codebases. To support this goal, we provide concrete mitigation strategies for each identified hotspot type, including semantic versioning, environment variables, linters, and cohesive refactoring planning.

By providing a tool to identify source code hotspots, categorizing their types, and documenting practical mitigation strategies, this research contributes to the broader understanding of software maintainability challenges while providing actionable guidance for creating more robust and stable software systems.

Data Availability. To facilitate reproducibility, we provide: our dataset of labeled, hotspots (including identified bots); the complete source code for line-tracking, hotspot detection, and analysis; accompanying scripts for statistical computations; and detailed repository-selection criteria along with full repository metadata. All these are available at <https://doi.org/10.5281/zenodo.15875147>.

References

- [1] Micah Allen, Davide Poggiali, Kirstie Whitaker, Tom Rhys Marshall, Jordy van Langen, and Rogier A. Kievit. 2021. Raincloud plots: a multi-platform tool for robust data visualization. *Wellcome Open Research* 4 (Jan. 2021), 63. doi:10.12688/wellcomeopenres.15191.2
- [2] Eric Allman. 2011. The robustness principle reconsidered. *Commun. ACM* 54, 8 (Aug. 2011), 40–45. doi:10.1145/1978542.1978557
- [3] Muhammad Asaduzzaman, Chanchal K. Roy, Kevin A. Schneider, and Massimiliano Di Penta. 2013. LHDiff: A Language-Independent Hybrid Approach for Tracking Source Code Lines. In *ICSM '13: Proceedings of the 29th IEEE International Conference on Software Maintenance*. IEEE, 230–239.
- [4] Marcel Böhme. 2018. STADS: Software testing as species discovery. *ACM Transactions on Software Engineering and Methodology* 27, 2 (2018), 1–52.
- [5] Hudson Borges and Marco Tulio Valente. 2018. What's in a GitHub Star? Understanding Repository Starring Practices in a Social Coding Platform. *Journal of Systems and Software* 146 (2018), 112–129. doi:10.1016/j.jss.2018.09.016
- [6] Pearl Brereton, Barbara A Kitchenham, David Budgen, Mark Turner, and Mohamed Khalil. 2007. Lessons from applying the systematic literature review process within the software engineering domain. *Journal of systems and software* 80, 4 (2007), 571–583. doi:10.1016/j.jss.2006.07.009
- [7] William H. Brown, Raphael C. Malveau, Hays W. McCormick, and Thomas J. Mowbray. 1998. *AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis*. John Wiley & Sons.
- [8] Jim Buckley, Tom Mens, Matthias Zenger, Awais Rashid, and Günter Kriesel. 2005. Towards a Taxonomy of Software Change. *Journal of Software Maintenance and Evolution: Research and Practice* 17, 5 (2005), 309–332. doi:10.1002/smr.319
- [9] Yuanfang Cai and Rick Kazman. 2019. DV8: Automated Architecture Analysis Tool Suites. In *TechDebt '19: Proceedings of the IEEE/ACM International Conference on Technical Debt*. IEEE/ACM, 53–54. doi:10.1109/TechDebt.2019.00015
- [10] Ozren Dabic, Emad Aghajani, and Gabriele Bavota. 2021. Sampling Projects in GitHub for MSR Studies. In *MSR '21: Proceedings of the 18th IEEE/ACM International Conference on Mining Software Repositories*. IEEE/ACM, 560–564. doi:10.1109/MSR52588.2021.00074
- [11] Tapajit Dey, Bogdan Vasilescu, and Audris Mockus. 2020. An Exploratory Study of Bot Commits. In *ICSE '20: Proceedings of the 42nd International Conference on Software Engineering Workshops*. IEEE/ACM, 61–65. doi:10.1145/3387940.3391502
- [12] Roberto Di Cosmo and Stefano Zacchiroli. 2017. Software heritage: Why and how to preserve software source code. In *iPRES 2017-14th International Conference on Digital Preservation*. 1–10.
- [13] Jamilah Din, Anas Bassam AL-Badareen, and Yusmadi Yah Jusoh. 2012. Antipatterns detection approaches in Object-Oriented Design: A literature review. In *2012 7th International Conference on Computing and Convergence Technology (ICCT)*. 926–931.
- [14] Thomas Dohmke. 2023. 100 Million Developers and Counting. *The GitHub Blog* 25 (2023).
- [15] Linda Erlenhov, Francisco Gomes De Oliveira Neto, and Philipp Leitner. 2020. An Empirical Study of Bots in Software Development: Characteristics and Challenges from a Practitioner's Perspective. In *ESEC/FSE '20: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, 445–455. doi:10.1145/3368089.3409680
- [16] Jean-Rémy Fallier and Matias Martinez. 2024. Fine-Grained, Accurate and Scalable Source Differencing. In *ICSE '24: Proceedings of the 46th International Conference*

- on Software Engineering. IEEE/ACM, 1–12.
- [17] Jean-Rémy Falleri, Flóreal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. 2014. Fine-Grained and Accurate Source-Code Differencing. In *ASE '14: Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering*. ACM/IEEE, 313–324.
 - [18] Beat Fluri, Emanuel Giger, and Harald C. Gall. 2008. Discovering Patterns of Change Types. In *ASE '08: Proceedings of the 23rd IEEE/ACM International Conference on Automated Software Engineering*. IEEE/ACM, 463–466. doi:10.1109/ASE.2008.74
 - [19] Beat Fluri, Michael Würsch, Martin Pinzger, and Harald Gall. 2007. Change Distilling: Tree Differencing for Fine-Grained Source Code Change Extraction. *IEEE Transactions on Software Engineering* 33, 11 (2007), 725–743. doi:10.1109/TSE.2007.70731
 - [20] Martin Fowler. 2018. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley Professional.
 - [21] Tatiana Fumasoli, Gaële Goastellec, and Barbara M. Kehm (Eds.). 2015. *Academic Work and Careers in Europe: Trends, Challenges, Perspectives*. Springer International Publishing. doi:10.1007/978-3-319-10720-2
 - [22] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. 1995. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, Reading, MA.
 - [23] Joshua Garcia, Daniel Popescu, George Edwards, and Nenad Medvidovic. 2009. Identifying Architectural Bad Smells. In *CSMR '09: Proceedings of the 13th European Conference on Software Maintenance and Reengineering*. IEEE, 255–258. doi:10.1109/CSMR.2009.59
 - [24] Emanuel Giger, Martin Pinzger, and Harald C. Gall. 2012. Can We Predict Types of Code Changes? An Empirical Analysis. In *MSR '12: Proceedings of the 9th IEEE Working Conference on Mining Software Repositories*. IEEE, 217–226.
 - [25] Mehdi Golzadeh, Alexandre Decan, Damien Legay, and Tom Mens. 2021. A Ground-Truth Dataset and Classification Model for Detecting Bots in GitHub Issue and PR Comments. *Journal of Systems and Software* 175 (2021), 110911. doi:10.1016/j.jss.2021.110911
 - [26] Todd L. Graves, Alan F. Karr, James S. Marron, and Harvey Siy. 2002. Predicting Fault Incidence Using Software Change History. *IEEE Transactions on Software Engineering* 26, 7 (2002), 653–661. doi:10.1109/32.859533
 - [27] Ahmed E. Hassan. 2009. Predicting Faults Using the Complexity of Code Changes. In *ICSE '09: Proceedings of the 31st International Conference on Software Engineering*. IEEE, 78–88. doi:10.1109/ICSE.2009.5070510
 - [28] A.S. Nazmul Huda and Soib Taib. 2013. Application of Infrared Thermography for Predictive/Preventive Maintenance of Thermal Defect in Electrical Equipment. *Applied Thermal Engineering* 61, 2 (Nov. 2013), 220–227. doi:10.1016/j.applthermaleng.2013.07.028
 - [29] Watts S. Humphrey. 1989. *Managing the Software Process*. Addison-Wesley, Reading, MA.
 - [30] International Organization for Standardization. 2025. *Information Technology — SoftWare Hash Identifier (SWHID) Specification V1.2*. Geneva, Switzerland. ISO/IEC 18670:2025.
 - [31] Md Rakibul Islam and Minhaz F. Zibran. 2021. What Changes in Where? An Empirical Study of Bug-Fixing Change Patterns. *ACM SIGAPP Applied Computing Review* 20, 4 (2021), 18–34. doi:10.1145/3447332.3447334
 - [32] Fehmi Jaafar, Angela Lozano, Yann-Gaël Guéhéneuc, and Kim Mens. 2017. Analyzing Software Evolution and Quality by Extracting Asynchrony Change Patterns. *Journal of Systems and Software* 131 (2017), 311–322. doi:10.1016/j.jss.2017.05.047
 - [33] Mohd Shawal Jadin and Soib Taib. 2012. Recent Progress in Diagnosing the Reliability of Electrical Equipment by Using Infrared Thermography. *Infrared Physics & Technology* 55, 4 (July 2012), 236–245. doi:10.1016/j.infrared.2012.03.002
 - [34] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. 2014. The Promises and Perils of Mining GitHub. In *MSR '14: Proceedings of the 11th Working Conference on Mining Software Repositories*. ACM, 92–101. doi:10.1145/2597073.2597074
 - [35] Brian W. Kernighan. 1979. *UNIX for Beginners*. Computer Science Technical Report 75. Bell Laboratories, Murray Hill, NJ. <https://web.archive.org/web/20170711222622/http://wolfram.schneider.org/bsd/7thEdManVol2/beginners/beginners.pdf> Available online <https://web.archive.org/web/20170711222622/http://wolfram.schneider.org/bsd/7thEdManVol2/beginners/beginners.pdf>.
 - [36] Sunghun Kim, Thomas Zimmermann, E. James Whitehead Jr., and Andreas Zeller. 2007. Predicting Faults from Cached History. In *ICSE '07: Proceedings of the 29th International Conference on Software Engineering*. IEEE, 489–498. doi:10.1145/1342211.1342216
 - [37] Timothy Kinsman, Mairieli Wessel, Marco A. Gerosa, and Christoph Treude. 2021. How Do Software Developers Use GitHub Actions to Automate Their Workflows?. In *MSR '21: Proceedings of the 18th IEEE/ACM International Conference on Mining Software Repositories*. IEEE/ACM, 420–431. doi:10.1109/MSR52588.2021.00054
 - [38] Andrew Koenig. 1998. Patterns and AntiPatterns. In *The Patterns Handbooks: Techniques, Strategies, and Applications*. Wiley, 383–389.
 - [39] Carsten Kolassa, Dirk Riehle, and Michel A. Salim. 2013. The empirical commit frequency distribution of open source projects. In *Proceedings of the 9th International Symposium on Open Collaboration (WikiSym '13)*. ACM, 1–8. doi:10.1145/2491055.2491073
 - [40] Chris Lamb and Stefano Zacchiroli. 2022. Reproducible Builds: Increasing the Integrity of Software Supply Chains. *IEEE Software* 39, 2 (March 2022), 62–70. doi:10.1109/ms.2021.3073045
 - [41] J. Richard Landis and Gary G. Koch. 1977. The Measurement of Observer Agreement for Categorical Data. *Biometrics* (1977), 159–174. doi:10.2307/2529310
 - [42] Steffen Lehnert and Matthias Riebisch. 2012. A Taxonomy of Change Types and Its Application in Software Evolution. In *ECBS '12: Proceedings of the 19th IEEE International Conference and Workshops on Engineering of Computer-Based Systems*. IEEE, 98–107. doi:10.1109/ECBS.2012.9
 - [43] Wei Lin, Zhifei Chen, Wanwangying Ma, Lin Chen, Lei Xu, and Baowen Xu. 2016. An Empirical Study on the Characteristics of Python Fine-Grained Source-Code Change Types. In *ICSME '16: Proceedings of the IEEE International Conference on Software Maintenance and Evolution*. IEEE, 188–199. doi:10.1109/ICSME.2016.25
 - [44] Danushka Liyanage, Marcel Böhme, Chakkrit Tantithamthavorn, and Stephan Lipp. 2023. Reachable coverage: Estimating saturation in fuzzing. In *ICSE '23: 45th IEEE/ACM International Conference on Software Engineering*. IEEE, 371–383.
 - [45] Tom Mens, Jim Buckley, Matthias Zenger, and Awais Rashid. 2003. Towards a Taxonomy of Software Evolution. In *IWUSE '03: Proceedings of the International Workshop on Unanticipated Software Evolution*. ACM.
 - [46] James Miller. 1999. Estimating the number of remaining defects after inspection. *Software Testing, Verification and Reliability* 9, 3 (1999), 167–189.
 - [47] W. Eugene Miller and W. Myers. 1985. A File Comparison Program. *Software: Practice and Experience* 15, 11 (1985), 1040–1044.
 - [48] Ran Mo, Yuanfang Cai, Rick Kazman, and Lu Xiao. 2015. Hotspot Patterns: The Formal Definition and Automatic Detection of Architecture Smells. In *2015 12th Working IEEE/IFIP Conference on Software Architecture*. IEEE, 51–60. doi:10.1109/wicsa.2015.12
 - [49] Ran Mo, Yuanfang Cai, Rick Kazman, Lu Xiao, and Qiong Feng. 2019. Architecture AntiPatterns: Automatically Detectable Violations of Design Principles. *IEEE Transactions on Software Engineering* 47, 5 (2019), 1008–1028. doi:10.1109/TSE.2019.2910856
 - [50] Ran Mo, Yuanfang Cai, Rick Kazman, Lu Xiao, and Qiong Feng. 2021. Architecture Anti-Patterns: Automatically Detectable Violations of Design Principles. *IEEE Transactions on Software Engineering* 47, 5 (May 2021), 1008–1028. doi:10.1109/tse.2019.2910856
 - [51] Audris Mockus and Lawrence Votta. 2000. Identifying Reasons for Software Changes Using Historic Databases. In *ICSM '00: Proceedings of the International Conference on Software Maintenance*. IEEE, 120–130. doi:10.1109/ICSM.2000.883028
 - [52] Nuthan Munaiah, Steven Kroh, Craig Cabrey, and Meiyappan Nagappan. 2017. Curating GitHub for Engineered Software Projects. *Empirical Software Engineering* 22 (2017), 3219–3253. doi:10.7287/peerj.preprints.2617v1
 - [53] Eugene W. Myers. 1986. An $O(ND)$ Difference Algorithm and Its Variations. *Algorithmica* 1, 1 (1986), 251–266.
 - [54] Nachiappan Nagappan and Thomas Ball. 2005. Use of Relative Code-Churn Measures to Predict System Defect Density. In *ICSE '05: Proceedings of the 27th International Conference on Software Engineering*. IEEE, 284–292. doi:10.1109/ICSE.2005.1553571
 - [55] Stas Negara, Mihai Codoban, Danny Dig, and Ralph E. Johnson. 2014. Mining Fine-Grained Code Changes to Detect Unknown Change Patterns. In *ICSE '14: Proceedings of the 36th International Conference on Software Engineering*. ACM, 803–813. doi:10.1145/2568225.2568317
 - [56] Jerzy Neyman. 1992. *On the Two Different Aspects of the Representative Method: the Method of Stratified Sampling and the Method of Purposive Selection*. Springer New York, 123–150. doi:10.1007/978-1-4612-4380-9_12
 - [57] Hoan Anh Nguyen, Anh Tuan Nguyen, Tung Thanh Nguyen, Tien N. Nguyen, and Hridesh Rajan. 2013. A Study of Repetitiveness of Code Changes in Software Evolution. In *ASE '13: Proceedings of the 28th IEEE/ACM International Conference on Automated Software Engineering*. IEEE/ACM, 180–190. doi:10.1109/ASE.2013.6693078
 - [58] Fabio Palomba, Gabriele Bavota, Massimiliano Di Penta, Rocco Oliveto, Andrea De Lucia, and Denys Poshyvanyk. 2013. Detecting Bad Smells in Source Code Using Change-History Information. In *ASE '13: Proceedings of the 28th IEEE/ACM International Conference on Automated Software Engineering*. IEEE/ACM, 268–278. doi:10.1109/ASE.2013.6693086
 - [59] Fabio Palomba, Gabriele Bavota, Massimiliano Di Penta, Rocco Oliveto, Denys Poshyvanyk, and Andrea De Lucia. 2014. Mining Version Histories for Detecting Code Smells. *IEEE Transactions on Software Engineering* 41, 5 (2014), 462–489. doi:10.1109/TSE.2014.2372760
 - [60] Fabio Palomba, Andrea De Lucia, Gabriele Bavota, and Rocco Oliveto. 2014. Anti-Pattern Detection. In *Advances in Computers*. Elsevier, 201–238. doi:10.1016/b978-0-12-800160-8.00004-8
 - [61] Fabio Palomba, Andy Zaidman, Rocco Oliveto, and Andrea De Lucia. 2017. An Exploratory Study on the Relationship Between Changes and Refactoring. In *ICPC '17: Proceedings of the 25th IEEE/ACM International Conference on Program Comprehension*. IEEE/ACM, 176–185. doi:10.1109/ICPC.2017.38

- [62] Petr Picha, Sebastian Hönel, Přemek Brada, Morgan Ericsson, Welf Löwe, Anna Wingkvist, and Jakub Daněk. 2022. Process anti-pattern detection – a case study. In *Proceedings of the 27th European Conference on Pattern Languages of Programs (EuroPLop '22)*. ACM, 1–18. doi:10.1145/3551902.3551965
- [63] Darius Sas, Paris Avgeriou, Ronald Kruijzinga, and Ruben Scheedler. 2020. Exploring the Relation Between Co-changes and Architectural Smells. *SN Computer Science* 2, 1 (Dec. 2020). doi:10.1007/s42979-020-00407-5
- [64] Tushar Sharma, Marios Fragkoulis, and Diomidis Spinellis. 2016. Does Your Configuration Code Smell?. In *MSR '16: Proceedings of the 13th IEEE/ACM Working Conference on Mining Software Repositories*. IEEE/ACM, 189–200. doi:10.1145/2901739.2901761
- [65] Tushar Sharma, Pratibha Mishra, and Rohit Tiwari. 2016. Designite: A Software Design Quality Assessment Tool. In *BADTDDA '16: Proceedings of the 1st International Workshop on Bringing Architectural Design Thinking into Developers' Daily Activities*. ACM, 1–4. doi:10.1145/2896935.2896938
- [66] Eric P. Smith and Gerald van Belle. 1984. Nonparametric Estimation of Species Richness. *Biometrics* (1984), 119–129.
- [67] Diomidis Spinellis. 2006. *Code Quality: The Open Source Perspective*. Addison-Wesley, Boston, MA. <https://www.spinellis.gr/codequality>
- [68] Diomidis Spinellis, Panos Louridas, and Maria Kechagia. 2021. Software Evolution: The Lifetime of Fine-Grained Elements. *PeerJ Computer Science* 7 (Feb. 2021), e372. doi:10.7717/peerj-cs.372
- [69] Diomidis Spinellis, Panos Louridas, Maria Kechagia, and Tushar Sharma. 2024. Broken Windows: Exploring the Applicability of a Controversial Theory on Code Quality. In *ICSME '24: International Conference on Software Maintenance and Evolution* (Flagstaff, AZ, USA). IEEE, 400–412. doi:10.1109/ICSME58944.2024.00044
- [70] Davide Taibi and Valentina Lenarduzzi. 2018. On the Definition of Microservice Bad Smells. *IEEE Software* 35, 3 (2018), 56–62. doi:10.1109/MS.2018.2141031
- [71] Alexander Trautsch, Johannes Erbel, Steffen Herbold, and Jens Grabowski. 2023. What Really Changes When Developers Intend to Improve Their Source Code: A Commit-Level Study of Static Metric Value and Static Analysis Warning Changes. *Empirical Software Engineering* 28, 2 (2023), 30. doi:10.1007/s10664-022-10257-9
- [72] Melina Vidoni. 2022. A Systematic Process for Mining Software Repositories: Results from a Systematic Literature Review. *Information and Software Technology* 144 (2022), 106791. doi:10.1016/j.infsof.2021.106791
- [73] Bruce F. Webster. 1995. *Pitfalls of Object-Oriented Development*. M&T Books.
- [74] Mairieli Wessel, Bruno Mendes De Souza, Igor Steinmacher, Igor S. Wiese, Ivanilton Polato, Ana Paula Chaves, and Marco A. Gerosa. 2018. The Power of Bots: Characterising and Understanding Bots in OSS Projects. *Proceedings of the ACM on Human-Computer Interaction* 2, CSCW (2018), 1–19. doi:10.1145/3274451
- [75] Xiaoya Xia, Zhenjie Weng, Wei Wang, and Shengyu Zhao. 2022. Exploring Activity and Contributors on GitHub: Who, What, When, and Where. In *2022 29th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 11–20. doi:10.1109/apsec57359.2022.00013